GERENCIADOR DE EVENTOS ACADÊMICOS: APLICAÇÃO WEB COM INTELIGÊNCIA ARTIFICIAL PARA INSCRIÇÕES, PRESENÇA E CERTIFICADOS

ACADEMIC EVENT MANAGER: A WEB APPLICATION WITH ARTIFICIAL INTELLIGENCE FOR REGISTRATION, ATTENDANCE AND CERTIFICATES

ALMEIDA, Bianca Gomes Dantas de[1]
MATOS, Jailton Santos de[1]
SANTOS, Lucas Neves dos[1]
SILVA, Giovanna Gomes[1]
SOUZA, Lucas Alves de[1]
MURAKAMI, Rafael Guem[2]

RESUMO

A organização de eventos acadêmicos costuma depender de planilhas, formulários avulsos e certificados produzidos manualmente, o que consome tempo e compromete a confiabilidade dos registros de participação. Este trabalho apresenta o desenvolvimento do Gerenciador de Eventos Acadêmicos, aplicação web que reúne em uma única plataforma a publicação de eventos, a inscrição com controle de vagas e fila de espera, o registro de presença, a emissão de certificados com código de autenticidade e a coleta de avaliações. Parte-se da hipótese de que a centralização dessas etapas, com regras de negócio verificadas automaticamente e com a inteligência artificial como apoio supervisionado ao cadastro, reduz o trabalho manual sem retirar do organizador o controle sobre o evento. Trata-se de uma pesquisa aplicada, conduzida por especificação de requisitos, modelagem, implementação incremental e verificação por testes automatizados de unidade, de integração e de ponta a ponta. A solução foi construída com Java e Spring Boot no servidor, React no navegador e banco de dados PostgreSQL, e publicada em serviços de nuvem gratuitos. Como resultado, obteve-se um sistema funcional que executa o fluxo completo, da criação do evento à validação pública do certificado, no qual o certificado é liberado apenas a quem teve presença registrada e após o término do evento, e no qual a falha do serviço de inteligência artificial não interrompe o cadastro. Conclui-se que a proposta atende aos objetivos traçados, embora a redução de tempo ainda não tenha sido medida com usuários reais.

Palavras-chave: Eventos acadêmicos. Inteligência artificial. Certificado digital. Desenvolvimento web.

---

[1] Graduandos em Análise e Desenvolvimento de Sistemas, turma 4A, da Universidade Santo Amaro – SP. Data da entrega: 25/10/2026.

[2] Professor orientador, Universidade Santo Amaro – SP.

ABSTRACT

Organizing academic events often relies on spreadsheets, scattered forms and manually produced certificates, which takes time and undermines the reliability of attendance records. This paper presents the development of the Academic Event Manager, a web application that brings together, in a single platform, event publishing, registration with seat control and a waiting list, attendance recording, the issuing of certificates with an authenticity code, and the collection of feedback. The hypothesis is that centralizing these steps, with automatically verified business rules and with artificial intelligence as supervised support for event registration, reduces manual work without taking control of the event away from the organizer. This is applied research, carried out through requirements specification, modeling, incremental implementation and verification by automated unit, integration and end-to-end tests. The solution was built with Java and Spring Boot on the server, React in the browser and a PostgreSQL database, and was deployed on free cloud services. As a result, a working system was obtained that runs the complete flow, from event creation to the public validation of the certificate, in which the certificate is released only to those whose attendance was recorded and only after the event ends, and in which a failure of the artificial intelligence service does not interrupt event registration. It is concluded that the proposal meets its objectives, although the time savings have not yet been measured with real users.

Keywords: Academic events. Artificial intelligence. Digital certificate. Web development.

1 INTRODUÇÃO

Palestras, workshops, semanas temáticas e congressos internos fazem parte da rotina das instituições de ensino superior e envolvem etapas que vão da divulgação à certificação dos participantes. Em muitos casos, essas etapas ainda são conduzidas de forma dispersa: as inscrições chegam por formulários avulsos, a presença é anotada em listas de papel ou em planilhas e os certificados são produzidos um a um, depois do evento. Além de consumir o tempo dos organizadores, esse arranjo dificulta saber quem de fato participou e permite que um certificado seja emitido sem comprovação de presença.

O problema que orienta este trabalho pode ser formulado da seguinte maneira: como reunir, em uma única ferramenta, as etapas de um evento acadêmico de modo que os registros de participação sejam confiáveis e que o esforço de cadastro e de certificação diminua? A hipótese adotada é que uma aplicação web que centralize essas etapas, com regras de negócio explícitas e verificadas automaticamente, e que empregue a inteligência artificial apenas como apoio supervisionado ao cadastro, reduz o trabalho manual sem retirar do organizador o controle sobre o evento.

O objetivo geral é desenvolver uma aplicação web para a gestão completa de eventos acadêmicos, capaz de administrar inscrições com controle de vagas e fila de espera, registrar a presença dos participantes e emitir certificados digitais com validação pública, incorporando recursos de inteligência artificial ao processo de cadastro. São objetivos específicos: oferecer controle de acesso diferenciado para participantes e organizadores; permitir a criação de eventos tanto por formulário quanto pela extração automática de dados a partir de texto livre; controlar a ocupação das vagas com fila de espera e promoção automática; emitir certificados vinculados à presença e verificáveis por qualquer pessoa; e garantir que a indisponibilidade do serviço de inteligência artificial não interrompa a operação.

A relevância do trabalho está na rastreabilidade que ele introduz. Quando o certificado depende de um registro de presença feito pelo organizador e pode ser conferido por um código público, a participação deixa de ser uma declaração e passa a ser um fato verificável. Ao mesmo tempo, a extração automática dos dados de um convite em texto livre reduz a etapa mais repetitiva do cadastro, sem dispensar a revisão humana antes da publicação.

Este artigo está organizado da seguinte forma: a subseção 1.1 descreve a metodologia empregada no desenvolvimento; a subseção 1.2 apresenta e discute os resultados, da especificação à publicação do sistema; e a seção 2 traz as considerações finais, as limitações e as sugestões de continuidade.

1.1 Metodologia

Trata-se de uma pesquisa de natureza aplicada, voltada à solução de um problema prático (GIL, 2010), com abordagem qualitativa e delineamento de desenvolvimento de software. O trabalho foi conduzido pelos cinco integrantes da equipe e organizado em oito etapas sucessivas: levantamento de requisitos e definição de escopo; modelagem na linguagem de modelagem unificada; modelagem de dados; prototipação e consolidação da documentação; configuração do ambiente; implementação do núcleo do sistema; presença, certificação e integração com inteligência artificial; e, por fim, testes, ajustes e apresentação.

Na primeira etapa, foram definidos dez requisitos funcionais, cinco requisitos não funcionais e onze regras de negócio, cada uma com identificador próprio. Esse

identificador acompanha a regra até o código e os testes: o trecho que a implementa a cita em comentário, e o teste correspondente a carrega no nome. A rastreabilidade assim obtida permitiu conferir, a cada mudança, se a especificação continuava atendida. A modelagem seguiu a especificação da linguagem de modelagem unificada (OBJECT MANAGEMENT GROUP, 2017) e produziu diagramas de casos de uso, classes, estados, atividades, interação e componentes, todos conferidos contra o código ao final do projeto.

A implementação adotou uma aplicação única dividida em camadas, com controladores, serviços e repositórios de responsabilidades separadas, padrão descrito por Fowler (2002) como forma de isolar a lógica de domínio da apresentação e do acesso a dados. O servidor foi escrito em Java 21 com Spring Boot 4.1, Spring Security e Spring AI; os dados ficam no PostgreSQL, com a estrutura criada por migrações versionadas do Flyway; e os certificados são montados com a biblioteca iText. A interface foi construída em React com TypeScript e Tailwind CSS. O controle de acesso usa tokens no padrão JSON Web Token (INTERNET ENGINEERING TASK FORCE, 2015), que dispensam o armazenamento de sessão no servidor. O código seguiu convenções de nomenclatura em português e funções de responsabilidade única, na linha recomendada por Martin (2008), e o histórico de versões adotou mensagens padronizadas.

A verificação baseou-se em testes automatizados organizados em pirâmide. Na base, testes de unidade com JUnit e Mockito validam as regras isoladamente; no meio, testes de integração sobem um banco PostgreSQL descartável em contêiner, por meio da biblioteca Testcontainers, e exercitam as rotas da aplicação; no topo, testes de ponta a ponta com Playwright percorrem a interface em um navegador real. A escrita dos testes acompanhou a implementação, prática que, segundo Beck (2002), faz dos testes um instrumento de projeto e não apenas de verificação. Concluída a implementação, a aplicação foi publicada em serviços gratuitos de nuvem, com o banco de dados no Neon, o servidor no Render e a interface na Vercel, e utilizada pela equipe em situação real, o que serviu como etapa adicional de validação.

O código-fonte, a especificação técnica e os diagramas estão disponíveis no repositório do projeto: https://github.com/Jayzmatos22/GerenciadorDeEventos.

1.2 Resultados e Discussão

1.2.1 Requisitos e regras de negócio

A especificação resultou em dez requisitos funcionais que cobrem o ciclo do evento: criação de conta e acesso, cadastro manual ou assistido por inteligência artificial, pesquisa no catálogo, inscrição com fila de espera, cancelamento, registro de presença, emissão de certificado e avaliação com resumo automático. Os requisitos não funcionais fixaram metas de desempenho, de resiliência diante da falha da inteligência artificial, de segurança das senhas e das chaves de acesso, de autorização por propriedade do evento e de funcionamento em telas de celular a partir de 360 pixels. As regras de negócio, resumidas no Quadro 1, concentram as decisões que mais afetam o comportamento do sistema.

Quadro 1 – Regras de negócio do sistema

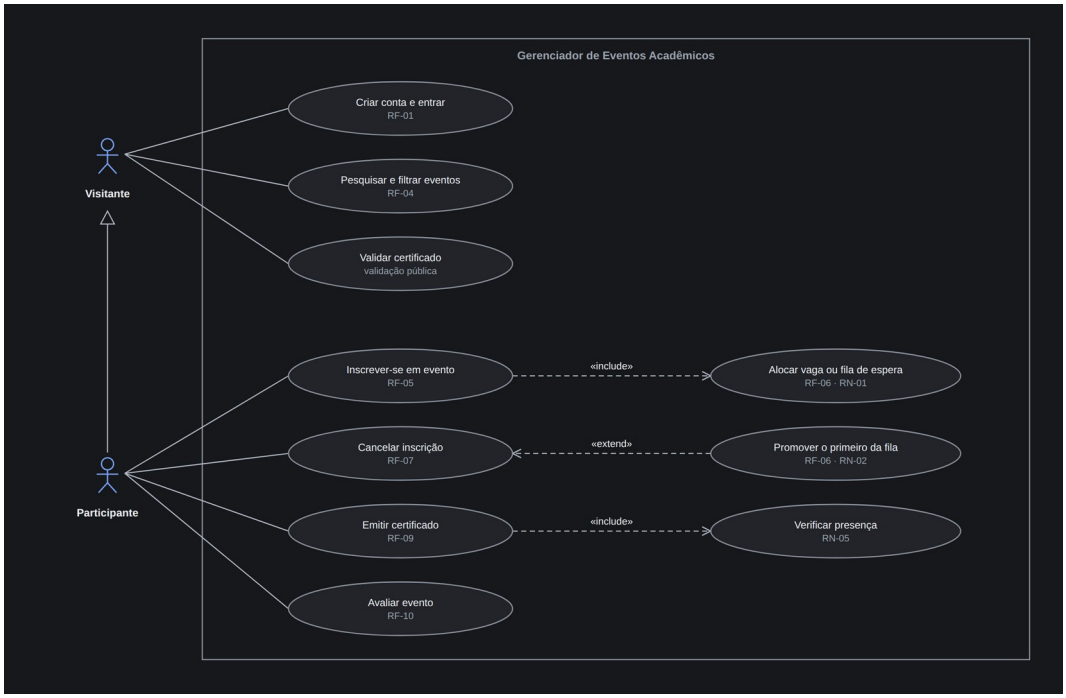
Regra	**Descrição resumida**
RN-01	Inscrição confirmada enquanto houver vaga; com o evento lotado, o participante entra no fim da fila de espera
RN-02	Cancelar uma inscrição confirmada promove automaticamente o primeiro da fila
RN-03	Uma única inscrição ativa por pessoa e evento; reinscrição permitida após cancelar
RN-04	Presença registrada só pelo organizador do evento, entre o início e 24 horas após o fim, para inscrições confirmadas
RN-05	Certificado só com presença registrada e após o término do evento, pelo horário ou pelo encerramento antecipado; emissão idempotente, com código de autenticidade
RN-06	Avaliação só após o fim do evento, por quem teve presença, uma por inscrição
RN-07	Resumo das avaliações por inteligência artificial só a partir de um mínimo de avaliações
RN-08	Somente o criador do evento pode gerenciá-lo
RN-09	Eventos encerrados ou cancelados não aceitam edição nem novas inscrições
RN-10	A falha da inteligência artificial não impede o cadastro manual
RN-11	Senhas com no mínimo 8 caracteres, guardadas apenas como resumo criptográfico

Fonte: Os autores (2026).

A Figura 1 apresenta os casos de uso do visitante e do participante. O participante herda as funcionalidades do visitante, como pesquisar eventos e validar certificados, e acrescenta as que dependem de conta. As relações de inclusão tornam obrigatórias a alocação de vaga na inscrição e a verificação de presença na emissão do certificado, enquanto a relação de extensão indica que a promoção do primeiro da fila só ocorre quando um cancelamento libera uma vaga. O organizador tem diagrama

próprio, no qual o preenchimento por inteligência artificial aparece como extensão da criação de evento.

Figura 1 – Diagrama de casos de uso do visitante e do participante

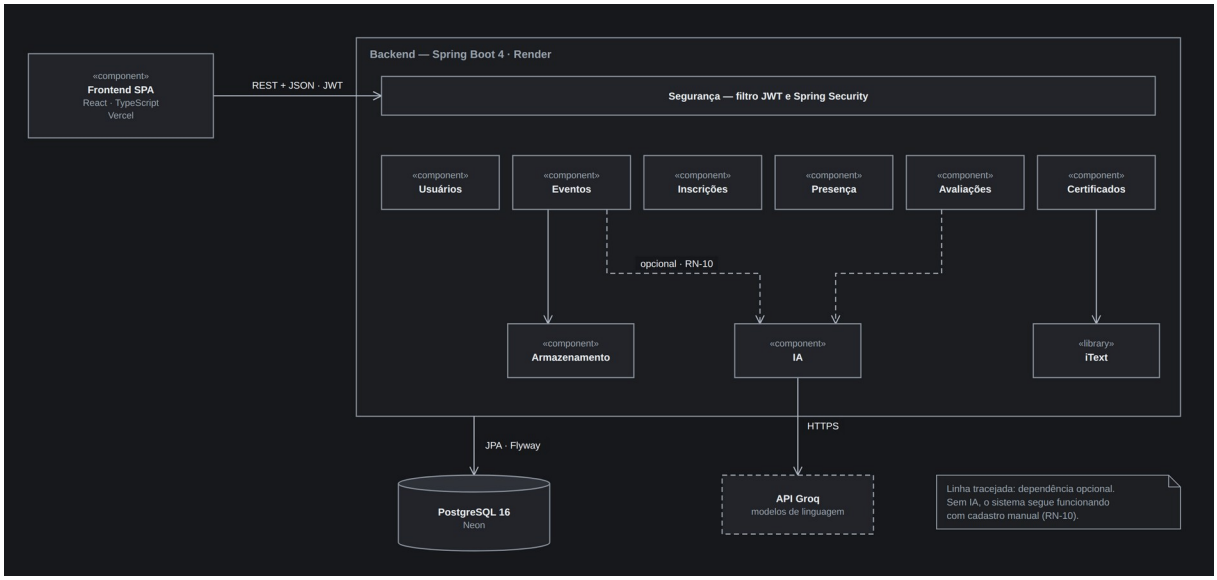


Fonte: Os autores (2026).

1.2.2 Arquitetura

A Figura 2 mostra a arquitetura adotada. A interface, publicada separadamente, comunica-se com o servidor por mensagens no formato JSON, e toda requisição passa por um filtro de segurança que valida o token antes de alcançar os módulos de domínio: usuários, eventos, inscrições, presença, avaliações e certificados. A separação entre interface e servidor permitiu publicar cada parte em uma plataforma especializada, ao custo de declarar explicitamente as origens autorizadas a consumir o servidor. As dependências do módulo de inteligência artificial aparecem tracejadas porque são opcionais: sem elas, o sistema continua operando com o cadastro manual.

Figura 2 – Diagrama de componentes



Fonte: Os autores (2026).

Uma decisão de projeto merece destaque. A verificação de que um usuário é dono do recurso que tenta alterar, como o organizador que edita um evento, é feita dentro dos serviços, e não apenas nas anotações de papel. O papel diz se a pessoa pode, em tese, executar a ação; a regra de propriedade decide se ela pode executá-la sobre aquele evento específico.

1.2.3 Inscrições e fila de espera

O controle de vagas exigiu atenção especial à concorrência. Quando duas pessoas disputam a última vaga ao mesmo tempo, as duas requisições calculam, de início, que há vaga disponível. A solução adotada usa o controle otimista de versão do evento: as duas transações tentam gravar a mesma versão, a perdedora falha e é repetida, já enxergando a vaga ocupada, e termina na fila de espera. Um teste de integração dispara duas threads simultâneas contra o mesmo evento e confirma esse comportamento. O cancelamento de uma inscrição confirmada promove o primeiro da fila na mesma transação e renumera as posições restantes, de modo que a fila nunca fica com lacunas.

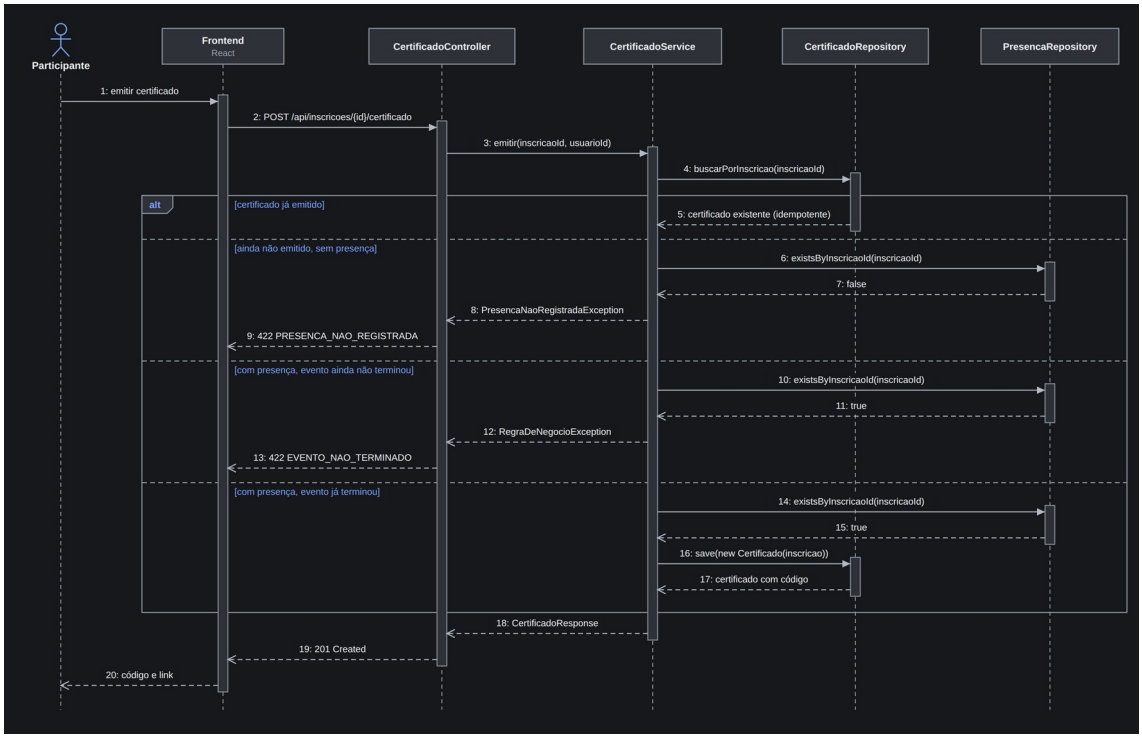
1.2.4 Presença e certificação

A presença é registrada pelo organizador durante o evento, e só é aceita para inscrições confirmadas, dentro da janela que vai do início até 24 horas após o término.

O registro é idempotente: uma segunda leitura do mesmo participante devolve a presença existente em vez de produzir erro, o que é útil quando o check-in é feito por leitura de código.

A regra de certificação foi a que mais evoluiu durante o projeto. Na versão inicial, a existência de presença bastava para a emissão. O uso do sistema publicado mostrou que isso permitia emitir o certificado logo após o check-in, antes de a pessoa ter permanecido no evento. A regra passou então a exigir também o término do evento, seja pelo horário de fim previsto, seja pelo encerramento antecipado feito pelo organizador; eventos cancelados não geram certificado. A Figura 3 apresenta o fluxo de emissão com essas condições.

Figura 3 – Diagrama de interação da emissão do certificado



Fonte: Os autores (2026).

Como mostra a Figura 3, a emissão é idempotente: um certificado já emitido é devolvido sem nova gravação, com o mesmo código de autenticidade. Sem presença, a resposta é a recusa por presença não registrada; com presença, mas antes do fim do evento, a recusa indica que o evento ainda não terminou. O documento em PDF não é gerado nesse momento, e sim a cada download, em memória e sem gravação em disco. A validação é pública: qualquer pessoa informa o código e recebe o nome do participante, o evento, a carga horária e as datas, sem dados de contato.

1.2.5 Inteligência artificial

A inteligência artificial é usada em dois pontos. No cadastro, o organizador cola o texto de um convite e o sistema devolve um rascunho com título, descrição, datas, local, carga horária e vagas. O recurso de saída estruturada do Spring AI anexa à requisição o esquema esperado, de modo que o modelo devolva dados organizados em vez de texto corrido. As instruções proíbem deduzir valores ausentes: o que o texto não informa volta vazio, em uma lista própria, e nada é gravado antes da revisão do organizador. No pós-evento, a inteligência artificial sintetiza as avaliações recebidas em um resumo consolidado, gerado apenas a partir de um número mínimo de avaliações.

Como o projeto utiliza serviços gratuitos, a resiliência foi tratada como requisito. A aplicação aceita uma lista de modelos de linguagem e a percorre até que um responda, iniciando de um ponto diferente a cada chamada; um modelo que a conta não pode usar é retirado da rotação. Cada chamada tem limite de 20 segundos e fica registrada, com ou sem sucesso, para auditoria. Esgotadas as tentativas, a interface informa a indisponibilidade e mantém o formulário manual intacto, o que atende à regra que impede a falha da inteligência artificial de bloquear o cadastro.

1.2.6 Testes, publicação e discussão

Ao final do projeto, a suíte do servidor reunia 136 testes automatizados de unidade e de integração, e a suíte de ponta a ponta somava onze cenários executados em navegador real, cobrindo o catálogo, a fila de espera, o fluxo completo do evento ao certificado, a indisponibilidade da inteligência artificial, a liberação do certificado após o encerramento e o uso de contas diferentes em abas do mesmo navegador. Os testes revelaram falhas que dificilmente seriam percebidas em verificações manuais. A primeira era de fuso horário: o servidor usava o relógio do sistema operacional, e, em um servidor no horário universal, as janelas de check-in e de avaliação ficariam deslocadas em três horas. A segunda era de ordem das respostas: uma busca antiga que chegasse depois da nova repunha a lista anterior e desfazia o filtro aplicado.

O uso do sistema publicado acrescentou três ajustes. Além da regra do certificado, já descrita, constatou-se que a sessão guardada de forma compartilhada entre as abas do navegador fazia uma conta substituir a outra quando organizador e

participante eram usados lado a lado; a sessão passou a ser guardada por aba. Verificou-se, ainda, que eventos em rascunho ou marcados como em andamento não aparecem no catálogo público, o que confundia quem os procurava; a tela do organizador passou a informar se o evento está visível.

Em relação à hipótese, os resultados indicam que a centralização foi alcançada: o fluxo que vai da criação do evento à validação pública do certificado é executado de ponta a ponta em uma única aplicação, e as regras que garantem a confiabilidade dos registros estão cobertas por testes. A inteligência artificial mostrou-se útil como apoio, desde que tratada como sugestão sujeita a revisão e cercada de um caminho alternativo. Por outro lado, a redução do tempo gasto pelos organizadores não foi medida com usuários reais, de modo que essa parte da hipótese permanece como expectativa fundamentada, e não como resultado comprovado.

2 CONCLUSÃO

O trabalho atingiu o objetivo de desenvolver uma aplicação web para a gestão completa de eventos acadêmicos. O sistema permite publicar eventos, controlar vagas com fila de espera, registrar a presença e emitir certificados verificáveis publicamente, e emprega a inteligência artificial no cadastro e no resumo das avaliações sem tornar a operação dependente dela. Aprendeu-se, sobretudo, que regras de negócio especificadas com precisão e verificadas continuamente sustentam a evolução do sistema: quando a regra do certificado precisou mudar, os testes que a descreviam apontaram o que precisava ser revisto.

Entre as limitações, destacam-se a ausência de medição do ganho de tempo com usuários reais, o registro de presença em um único momento, que não comprova a permanência do participante até o fim, e o armazenamento apenas temporário das fotos de divulgação no plano gratuito de hospedagem. Ficaram fora do escopo a cobrança de inscrições, a divisão do evento em sessões, a submissão de trabalhos e as notificações por correio eletrônico.

Como continuidade, sugere-se avaliar o sistema com organizadores e participantes em eventos reais, medindo o tempo de cadastro e de certificação; registrar também a saída dos participantes, para comprovar a permanência;

armazenar as imagens em serviço próprio para objetos; e incluir o envio de notificações sobre inscrições, promoções da fila e disponibilidade do certificado.

REFERÊNCIAS

BECK, Kent. Test-driven development: by example. Boston: Addison-Wesley, 2002.

FOWLER, Martin. Patterns of enterprise application architecture. Boston: Addison-Wesley, 2002.

GIL, Antonio Carlos. Como elaborar projetos de pesquisa. 2. ed. São Paulo: Atlas, 2010.

INTERNET ENGINEERING TASK FORCE. RFC 7519: JSON Web Token (JWT). [S. l.]: IETF, 2015. Disponível em: https://datatracker.ietf.org/doc/html/rfc7519. Acesso em: 25 out. 2026.

MARTIN, Robert C. Clean code: a handbook of agile software craftsmanship. Upper Saddle River: Prentice Hall, 2008.

OBJECT MANAGEMENT GROUP. OMG Unified Modeling Language (OMG UML): version 2.5.1. Needham: OMG, 2017. Disponível em: https://www.omg.org/spec/UML/. Acesso em: 25 out. 2026.